90-Day Career Switch Roadmap

From Enterprise Data/Support Experience to AI-Powered Backend / GenAI Engineering

Target: 18 LPA+ | Timeline: 12 weeks | Core direction: Python + Backend + GenAI + Cloud + System Design + DSA

1. The Career Strategy

The recommended transition is not to restart from zero as a pure ML engineer. Instead, build on existing enterprise experience and move toward AI Application Engineering / GenAI Engineering / Backend Engineering with AI.

Core positioning:

Software Engineer who builds AI-powered systems.

This uses existing strengths in production support, SQL, microservices, debugging, enterprise systems, cloud/data-platform exposure and Python, while adding LLMs, RAG, agents, APIs, cloud deployment and AI system design.

2. Target Roles

- AI / GenAI Engineer
- AI Application Engineer / LLM Engineer
- Backend Engineer + AI / GenAI
- Python Backend Developer
- Applied AI Engineer
- AI Platform Engineer
- Software Engineer – GenAI
- Python AI Engineer

Highest-priority search pattern: Python + FastAPI + LLM + RAG + APIs + microservices + cloud.

3. Why This Route Fits

A pure ML Engineer path would require compressing deep learning, statistics, model training, PyTorch/TensorFlow and broader ML foundations into a short period. A GenAI application/backend path is a more natural extension of production engineering experience.

The desired profile becomes:

Production engineering → backend development → AI-powered backend engineering

4. Skill Stack

- Core: Python, SQL, Git, Linux
- Backend: FastAPI, REST APIs, microservices, PostgreSQL, Redis, Docker
- AI: LLM APIs, RAG, embeddings, vector databases, tool calling, AI agents, LLM evaluation
- Cloud: Pick one platform; AWS is a strong default

- Interview: DSA, system design, SQL, CS fundamentals

5. The 12-Week Roadmap

Weeks 1–2 — Python & Backend

- Week 1: Python — OOP, decorators, generators, iterators, comprehensions, typing, dataclasses, exceptions, logging, pytest, asyncio/concurrency basics.
- Start DSA immediately: arrays, strings, hashmaps, sets, two pointers, sliding window, stacks and queues.
- Week 2: HTTP, REST, JSON, status codes, authentication/JWT, pagination, validation, then FastAPI, Pydantic, dependency injection, middleware, async endpoints and OpenAPI.
- Mini Project #1: Production Ticket API using FastAPI + PostgreSQL. Add authentication, logging, validation, error handling and Docker.

Weeks 3–4 — Backend, Data & Cloud

- Week 3: PostgreSQL fundamentals — indexes, joins, transactions, normalization, query plans, ACID, isolation and connection pooling.
- Learn Redis for caching, TTL, sessions and rate limiting.
- Learn Kafka fundamentals: producers, consumers, topics, partitions, offsets and consumer groups.
- Week 4: Docker — images, containers, volumes, networks, Dockerfiles and Compose.
- Pick one cloud. AWS is a strong choice. Learn EC2, S3, IAM, Lambda, RDS and CloudWatch at a practical level.

Weeks 5–6 — GenAI Foundations

- Week 5: LLM fundamentals — tokens, context windows, temperature, system prompts, structured outputs, embeddings, inference, hallucinations, model selection and function/tool calling.
- Mini Project #2: AI Ticket Classifier that converts a ticket into structured category, severity, service, summary and recommended action.
- Week 6: RAG — document ingestion, chunking, embeddings, vector search, metadata, top-k retrieval, reranking and context construction.
- Use PostgreSQL + pgvector initially; it connects naturally to existing SQL experience.
- Project #3: Enterprise Knowledge Assistant that answers engineering/runbook questions with source-backed retrieval.

Weeks 7–8 — Tool Calling & Agents

- Week 7: Function/tool calling — LLM decides which backend function to call, receives the result and produces an interpreted response.
- Example tools: get_service_health(), get_failed_transactions(), get_recent_deployments(), search_runbooks(), get_previous_incidents().
- Week 8: AI agents — learn state, workflows, tool use, retries, guardrails, human-in-the-loop and structured outputs. LangGraph can be used for agent workflows.
- Do not over-focus on multi-agent hype; one reliable agent/workflow is more valuable than a complicated demo.

Weeks 9–10 — Major Portfolio Project

- Build the flagship project: AI Production Incident Investigator.
- Concept: combine PagerDuty/Jira-style incident workflows, SQL analysis, RAG, LLMs and tool calling.

- Architecture: User → FastAPI → AI Orchestrator → SQL Tool / RAG Tool / API Tool → PostgreSQL / pgvector / service APIs → LLM → Incident Report.
- Example request: “Why did payment failures increase between 2–3 PM?”
- The system should query data, inspect error spikes, retrieve runbooks, check deployments, compare previous incidents, correlate evidence and produce a likely root cause with recommended actions.
- Add natural-language-to-SQL with validation, a read-only database and an allow-list of tables/columns.
- Target output: Incident, Impact, Likely Root Cause, Evidence, Confidence and Recommended Actions.

Weeks 11–12 — Interview Mode

- DSA target: about 100 quality problems, prioritizing patterns over volume.
- Cover arrays, strings, hashmaps, two pointers, sliding window, stack/queue, linked lists, binary search, trees, heaps, graphs, recursion/backtracking and basic dynamic programming.
- System design: URL shortener, notification system, file upload service, rate limiter and chat system.
- AI system design: RAG architecture, ingestion, chunking, embedding, retrieval, reranking, LLM, caching, latency, cost, security, evaluation and monitoring.
- Begin applying around Weeks 8–9 rather than waiting for Day 90.

6. Daily Schedule

Monday–Friday: 2.5–3 hours/day

- 45 min — DSA
- 60 min — Learning
- 60 min — Project

On busy days, aim for a 90-minute minimum rather than abandoning the week.

Saturday: 5–6 hours — 2h project, 1.5h DSA, 1h system design, 30m revision.

Sunday: 4–5 hours — project, interview prep, revision, resume/LinkedIn and applications.

7. Portfolio by Day 90

- Project 1: Production Ticket Management API — Python + FastAPI + PostgreSQL + Redis + Docker
- Project 2: Enterprise RAG Assistant — Python + FastAPI + pgvector + LLM
- Project 3: AI Production Incident Investigator — Python + FastAPI + PostgreSQL + pgvector + LLM + tool calling/agent workflow + Docker + AWS

The third project should be the centerpiece because it connects enterprise production experience with modern AI engineering.

8. Resume Positioning

Do not present the profile as simply “Data Engineer.” Emphasize enterprise production experience, microservices, backend development, Python, SQL, cloud and the new GenAI portfolio.

- Support experience can be framed as production microservice support, root-cause analysis, service-level diagnostics and issue resolution.
- Cloudera/data-platform exposure can be described as enterprise data-platform and distributed data-processing experience.

- Development work should emphasize Python/SQL components and microservice/application development.
- New portfolio work demonstrates the transition into AI-powered software engineering.

Suggested headline: Software Engineer | Python | Backend | GenAI | RAG | LLM Applications

9. Interview Questions to Master

- What is RAG and how does the architecture work?
- Why RAG instead of fine-tuning?
- How do embeddings work?
- How do you reduce hallucinations?
- How would you evaluate an LLM application?
- How do you protect against prompt injection?
- How do you safely execute LLM-generated SQL?
- How would you reduce LLM latency and cost?
- How would you handle 1 million documents?
- How would you monitor a production LLM application?

10. Job Search Strategy

- Week 6: Start monitoring target jobs and identify recurring skill requirements.
- Week 8: Begin selective applications.
- Week 10: Apply aggressively while continuing interview preparation.
- Week 12: Interview heavily and iterate based on feedback.

Search job titles such as GenAI Engineer, Generative AI Engineer, AI Application Engineer, LLM Engineer, GenAI Application Developer, Python AI Engineer, AI Backend Engineer, Backend Engineer – GenAI, Software Engineer – GenAI, Python Backend Developer – AI and Applied AI Engineer.

11. What Not to Do

- Do not attempt to become a deep-learning/ML expert from scratch in three months.
- Do not spend most of the period collecting certificates.
- Do not build only generic PDF chatbots or shallow chatbot demos.
- Do not wait until Day 90 to apply.
- Do not quit the current job before securing the next offer.

Prioritize projects + interview preparation + positioning over certificates.

12. Final Target Profile

By the end of 90 days, the ideal profile looks like:

4 YOE enterprise experience → production/microservices → Python/backend → cloud → GenAI → LLM APIs → RAG → vector search → tool calling → agents → AI system design.

The career narrative becomes:

“I am a software engineer with enterprise production experience who builds AI-powered systems.”

The 18 LPA+ goal is ambitious but should be treated as a target for roles whose market compensation supports it, rather than assuming the current 8 LPA package determines the next package.